Desarrollo de un compilador con Flex, Bison y LLVM

Para esta tarea tenía que hacer un compilador que leyera un lenguaje sencillo y generara código para el Arduino Mega 2560. Lo dividí en varias etapas, hasta llegar al firmware final.

Primero, usé Flex para el análisis léxico. Flex lee el archivo fuente y va separando las palabras en piezas con sentido, a las que llamamos tokens. Por ejemplo, reconoce let y print como palabras reservadas, identifica los nombres de variables y los números, y también los símbolos como el igual o el punto y coma.

Bison que es el encargado del análisis sintáctico. Él recibe los tokens y revisa que estén acomodados según las reglas de la gramática. Mientras validaba cada instrucción, también fui armando el AST, que es como un árbol que muestra la estructura del programa y guarda las instrucciones y expresiones de forma que luego es fácil recorrerlas.

La siguiente parte fue el análisis semántico, aquí recorrí el AST para revisar que el programa no solo estuviera escrito bien, si no que tuviera sentido. Usé una tabla de símbolos donde guardo cada variable con su tipo, y así puedo detectar cosas como usar una variable que nunca se declaró. Si hay un error, el compilador lo muestra y se detiene antes de generar código.

Cuando el programa pasa todas las revisiones, se genera LLVM IR, que es una representación intermedia en texto. Ese código lo verifiqué con las herramientas de LLVM, y luego llc lo convirtió a ensamblador AVR para el ATmega2560. Por último, avr-gcc se encargó de ensamblar y enlazar todo con un runtime de prueba, y así obtuve el firmware en formato ELF y HEX.

Hacer todo este proceso fue para entender cómo funciona un compilador a este nivel de bajo nivel y sé que dominar esto es clave si algún día quiero trabajar en Google o en empresas tecnológicas grandes creando tecnología de verdad.